

CÍVICA SOFTWARE

Sistema de gestión de incidencias · Turno de recuperación

Cuadernillo del desarrollador de turno

Prueba práctica de recuperación · Corte 1 · Algoritmos y estructura de datos

Nombre completo	Edward Castillo Rebolledo	Fecha	2/09/2026
-----------------	---------------------------	-------	-----------

Este cuadernillo contiene todo lo que usted necesita durante el turno: la mecánica de la prueba, las instrucciones detalladas de los tres tickets y su hoja de ruta. Consérvelo: es el documento que se entrega en la mesa de despacho al cerrar el turno.

Cómo funciona este turno

Este es un turno de recuperación. Son tres tickets en lugar de cinco, pero cubren los mismos temas del Corte 1: arreglos y matrices, memoria dinámica y punteros, programación orientada a objetos, y listas enlazadas.

La mecánica es la misma que ya conoce: un ticket queda cerrado cuando su código pasa las pruebas automáticas, y al pasar aparece un código de cierre que usted anota en la hoja de ruta.

Momento	Minuto	Actividad	Duración
1	0 – 10	Briefing del turno y revisión de la carpeta de trabajo	10 min
2	10 – 35	TCK-5510 · P3 — Mapa de cobertura con datos incompletos	25 min
3	35 – 65	TCK-5511 · P2 — Registro de sanciones: memoria y tipos	30 min
4	65 – 90	TCK-5512 · P0 — Historial corrupto: producción caída	25 min

Duración total del turno: 90 minutos. Las ventanas son una referencia de ritmo, no un plazo: si cierra un ticket después de que su ventana pasó, cuenta igual.

El ciclo de trabajo

1. **Lea el ticket completo:** el reporte de quien lo abrió y el criterio de aceptación.
2. **Corrija únicamente el archivo indicado** en la ficha del ticket.
3. **Ejecute las pruebas automáticas.** Le muestran qué pasa y qué falla. Puede ejecutarlas cuantas veces quiera.
4. **Cuando todas pasen, aparece el CÓDIGO DE CIERRE.** Anótelos en su hoja de ruta.

SOBRE EL CÓDIGO DE CIERRE No está escrito en ningún archivo: se calcula a partir de los resultados correctos. No se puede adivinar, ni leer del archivo de pruebas, ni copiar del compañero.

Sus archivos y sus comandos

```
prueba_recuperacion/estudiante/
TCK-5510/  cobertura.py  pruebas_t1.py
TCK-5511/  sanciones.cpp  pruebas_t3.py
TCK-5512/  historial.py
```

Ticket	Usted modifica	Usted ejecuta
TCK-5510	cobertura.py	python3 pruebas_t1.py
TCK-5511	sanciones.cpp	g++ -std=c++17 -fsanitize=address -g -o sanciones sanciones.cpp ./sanciones
TCK-5512	historial.py	python3 pruebas_t3.py

REGLA QUE NO TIENE EXCEPCIÓN Los archivos cuyo nombre empieza por «pruebas» NO se modifican. Alterarlos para que impriman el código es fraude y aplica el protocolo del reglamento.

Su nota

La nota es la suma de los tickets que cierre. Los tickets son independientes: puede resolverlos en cualquier orden y saltarse el que se le atragante. Puede cerrar su turno y salir cuando quiera.

Ticket	Sev	Vale	Si cierra hasta aquí y sale	Resultado
TCK-5510	P3	1.5	1.5	Insuficiente
TCK-5511	P2	1.7	3.2	Aprueba
TCK-5512	P0	1.8	5.0	Turno completo

- **El TCK-5511 es el que decide la aprobación.** Con los dos primeros cerrados usted ya aprueba.
- **El TCK-5512 vale más que los otros.** Si le queda poco tiempo y no ha cerrado nada, es el que más rinde.

La mesa de despacho

1. Cuando decida cerrar su turno, lleve este cuadernillo a la mesa.
2. La profesora le pedirá que ejecute las pruebas en su pantalla, delante de ella. No sirve una captura ni una foto.
3. Coteja los códigos, firma su hoja de ruta y usted se retira.

Reglas del turno

- Trabajo individual. Cada uno en su equipo.
- Material permitido: únicamente la documentación oficial de C++ y de Python.
- Puede ejecutar las pruebas cuantas veces quiera. No hay penalización por intentar.
- Si su código no compila, lea el error completo antes de levantar la mano.
- Una vez firma en la mesa, el turno terminó: no se puede volver a entrar.

Nota: del primer parcial quiero reemplazar el ticket TCK-4421, TCK-4422

TCK-5510 El mapa de cobertura está incompleto**SEVERIDAD P3** · RedAcopio · ventana 10–35 min · Python · 1.5 puntos

Reportado por: Coordinación de rutas de recolección

«El informe de cobertura por zona nunca muestra la última zona del barrio. Siempre sale una columna de menos.»

«Y necesitamos dos datos que nunca se implementaron: cuál ruta es la más productiva, y cuántas zonas quedaron sin cubrir del todo.»

Contexto del sistema

El archivo cobertura.py contiene una matriz donde las filas son rutas del camión y las columnas son zonas del barrio. Cada celda es lo recogido por esa ruta en esa zona. La matriz y el archivo de pruebas no se modifican.

Qué debe hacer

1. **Corregir cobertura_por_zona(m)**. Debe devolver el total de CADA columna, incluida la última. Actualmente el recorrido se detiene una posición antes: revise el límite del ciclo. -
2. **Implementar ruta_mas_productiva(m)**. Debe devolver el ÍNDICE (no el valor) de la fila que más kilos recogió en total. -
3. **Implementar zonas_sin_cubrir(m)**. Debe devolver cuántas COLUMNAS quedaron completamente en cero, es decir, zonas donde ninguna ruta recogió nada. Ojo: no es contar ceros sueltos, es contar columnas enteras vacías. -

Criterio de aceptación

- Las cuatro verificaciones de pruebas_t1.py en OK.
- No cambie los nombres ni los parámetros de las funciones: el archivo de pruebas las llama por su nombre.
- No modifique la matriz cobertura.

Cómo se verifica

```
$ cd prueba_recuperacion/estudiante/TCK-5510
$ python3 pruebas_t1.py
```

Errores frecuentes en este ticket

- Devolver el valor máximo en vez del índice, en ruta_mas_productiva.
- Contar ceros individuales en vez de columnas completas, en zonas_sin_cubrir. Una columna solo cuenta si TODAS sus celdas son cero.
- Confundir cuál de los dos ciclos va por fuera al recorrer por columnas.

Código de cierre obtenido: _____

```

7  # filas = rutas del camion, columnas = zonas del barrio
8  # cada celda = kilos recogidos por esa ruta en esa zona
9  cobertura = [
10     [5, 0, 3, 0, 2, 4, 0],
11     [0, 0, 7, 0, 1, 0, 0],
12     [2, 0, 0, 0, 4, 3, 1],
13     [0, 0, 5, 0, 0, 0, 2],
14 ]
15
16 def total_por_ruta(m):
17     """Devuelve una lista con el total recogido por cada ruta (fila)."""
18     totales = []
19     for fila in m:
20         s = 0
21         for v in fila:
22             s += v
23         totales.append(s)
24     return totales
25

```

```
def cobertura_por_zona(m):
    """Devuelve una lista con el total recogido en cada zona (columna).
    BUG REPORTADO: la ultima zona nunca aparece en el informe."""
    totales = []
    for j in range(len(m[0])):          #se quitó el -1 para que recorra hasta la última columna
        s = 0
        for i in range(len(m)):
            s += m[i][j]
        totales.append(s)
    return totales

def ruta_mas_productiva(m):
    """Devuelve el INDICE de la ruta que mas kilos recogio en total."""
    totales = total_por_ruta(m)         #llama a la funcion total_por_ruta para obtener los totales de cada ruta
    max_kilos = max(totales)            #usando max() porque es un metodo nativo de python y es mas eficiente que recorrer la lista con un loop
    return totales.index(max_kilos)     #devuelve el indice de la ruta con el máximo total
```

```
47 def zonas_sin_cubrir(m):
48     """Devuelve cuantas zonas (columnas) quedaron COMPLETAMENTE en cero,
49     es decir, ninguna ruta recogio nada allí."""
50     totales = cobertura_por_zona(m)    #llama a la funcion cobertura_por_zona para obtener los totales de cada zona
51     return totales.count(0)            #devuelve la cantidad de zonas que quedaron en cero usando count()
52                                     #porque es un metodo nativo de python y es mas eficiente que recorrer la lista con un loop
53
54 print(total_por_ruta(cobertura))
55 print(cobertura_por_zona(cobertura))
56 print("la ruta mas productiva es la numero:", ruta_mas_productiva(cobertura))
57 print("las zonas sin cobertura son:", zonas_sin_cubrir(cobertura))
```

```
[14, 14, 10, 15]
[7, 0, 15, 0, 7, 15, 9]
la ruta mas productiva es la numero: 3
las zonas sin cobertura son: 2
TCK-5510 · verificacion
[OK ] total_por_ruta: obtuvo [14, 14, 10, 15]
[OK ] cobertura_por_zona: obtuvo [7, 0, 15, 0, 7, 15, 9]
[OK ] ruta_mas_productiva: obtuvo 3
[OK ] zonas_sin_cubrir: obtuvo 2

TICKET CERRADO – codigo de cierre: 5510-5332
PS C:\Users\veecg\OneDrive\Documents\GitHub\Eduard-Castillo>
```

En la línea 31 se hizo un cambio comentado en el código

En la línea 41 se agregó un método a la función, comentado en el código, para que pudiera funcionar

En la línea 50 se agregó un método a la función, comentado en el código, para que pudiera funcionar

desde la línea 54 a 57 se agregó la función print() para mostrar los resultados de lo que nos traen las funciones

TCK-5511 Registro de sanciones: no distingue tipos y consume memoria**SEVERIDAD P2** · PrestaLab · ventana 35–65 min · C++ · 1.7 puntos

Reportado por: Coordinación de préstamos e Infraestructura

«El listado imprime «Usuario» para todos. Necesitamos distinguir estudiantes de usuarios externos.»

«Las sanciones del primer incidente de cada usuario no se están contando: el total sale corto.»

«Y el proceso arranca en 40 MB y termina el día en 600 MB. Hay que reiniciarlo cada noche.»

Contexto del sistema

El archivo sanciones.cpp gestiona el registro de sanciones. Cada objeto Usuario reserva memoria dinámica para su arreglo de días de sanción. Hay una clase base y faltan dos clases especializadas.

SON TRES PROBLEMAS DISTINTOS Este ticket combina lógica, polimorfismo y memoria. Resuelva primero la lógica y las clases: son las que destraban el código de cierre. La memoria se verifica aparte, con el sanitizer.

Qué debe hacer

1. **Corregir el defecto de lógica en totalDias().** El recorrido empieza en la posición 1 y por eso se pierde el primer incidente de cada usuario.
2. **Crear la clase Estudiante,** que herede de Usuario y agregue el programa académico. Su descripción debe devolver exactamente: Estudiante ES-002 de Ingeniería
3. **Crear la clase Externo,** que herede de Usuario y agregue la entidad de procedencia. Su descripción debe devolver exactamente: Externo EX-003 (Alcaldía)
4. **Activar las líneas comentadas de main,** tanto las que crean los objetos como las que registran sus sanciones.
5. **Hacer que cada objeto responda con SU propia descripción** aunque se recorra el arreglo con punteros a la clase base. Si todos imprimen «Usuario», falta una palabra clave.
6. **Cerrar las fugas de memoria.** Busque los comentarios FALTA ALGO AQUI y FALTAN LIBERACIONES AQUI. Recuerde que una clase base con clases hijas necesita destructor virtual.

Criterio de aceptación

- El programa imprime las tres descripciones, cada una con su formato propio.
- El programa imprime TOTAL=12 y a continuación el código de cierre.
- Compilado con -fsanitize=address, la ejecución NO produce ningún bloque ERROR: AddressSanitizer.
- Se verifican las dos últimas cosas en la mesa de despacho. Una sola no basta.

Cómo se verifica

```
$ cd prueba_recuperacion/estudiante/TCK-5511
$ g++ -std=c++17 -fsanitize=address -g -o sanciones sanciones.cpp
$ ./sanciones
```

El texto de las descripciones se compara carácter por carácter. Respete mayúsculas, espacios y paréntesis exactamente como se indican arriba.

Errores frecuentes en este ticket

- Olvidar virtual en descripcion(): todos los objetos imprimen la descripción de la clase base.
- Olvidar el destructor virtual: el sanitizer reporta fuga al liberar por puntero a la base.
- Liberar el arreglo de punteros antes que los objetos que contiene: se pierden las direcciones.
- Activar la creación de los objetos pero olvidar activar las líneas que registran sus sanciones: el total queda corto.

Código de cierre obtenido: _____

TCK-5512 PRODUCCIÓN CAÍDA — el historial está corrupto**SEVERIDAD PO** · Turno justo · ventana 65–90 min · Python · 1.8 puntos

Reportado por: Mesa de ayuda · tres reportes en la última hora

«Registré la primera atención del día y el sistema se cayó.»

«Deshice la última atención y se borró todo el historial.»

«Busco un turno que sí existe y me dice que no está.»

Contexto del sistema

El archivo historial.py implementa el historial de atenciones de un punto de servicio mediante una lista enlazada simple. Cada nodo guarda un número de turno y el módulo que lo atendió. Los métodos cuantas y listar funcionan bien.

Qué debe hacer

1. **Corregir registrar().** Se cae cuando el historial está vacío, porque intenta recorrer desde una cabeza que no existe. Falta el caso de la lista vacía: cuando no hay cabeza, el nuevo nodo ES la cabeza.-
2. **Corregir deshacer_ultima().** Debe eliminar únicamente el ÚLTIMO nodo, no todo el historial. Recuerde que para desenganchar el último hay que llegar al PENÚLTIMO. Y maneje aparte el caso de un solo elemento, donde sí se vacía el historial.-
3. **Implementar buscar(turno).** Debe recorrer el historial y devolver el MÓDULO que atendió ese turno, o None si el turno no existe.-

NO CAMBIE LAS FIRMAS Los métodos deben seguir llamándose registrar, deshacer_ultima y buscar, y recibir los mismos parámetros. El archivo de pruebas los llama por su nombre.

Criterio de aceptación

- Las ocho verificaciones de pruebas_t3.py en OK.
- Registrar en un historial vacío funciona y no lanza excepción.
- Deshacer la última atención deja el resto del historial intacto.
- Deshacer con un solo elemento deja el historial en cero.
- Deshacer en un historial vacío devuelve False sin lanzar excepción.
- Buscar un turno existente devuelve su módulo; buscar uno inexistente devuelve None.

Cómo se verifica

```
$ cd prueba_recuperacion/estudiante/TCK-5512
$ python3 pruebas_t3.py
```

EL VALIDADOR NO SE CAE Si su código lanza una excepción, el archivo de pruebas la captura y la reporta como FALLA, y sigue con las demás verificaciones. Así usted ve el panorama completo en cada ejecución.

Sugerencia de método

Antes de tocar el código, dibuje en el espacio de abajo cuatro nodos enlazados y marque con una flecha cuál referencia debe cambiar al eliminar el último. Después dibuje el caso de un solo nodo. Este ticket se resuelve más rápido dibujando que probando.

Código de cierre obtenido: _____

```

11 class Nodo:
12     def __init__(self, turno, modulo):
13         self.turno = turno
14         self.modulo = modulo
15         self.siguiente = None
16
17
18 class Historial:
19     def __init__(self):
20         self.cabeza = None
21
22     def registrar(self, turno, modulo):
23         """Agrega una atencion al FINAL del historial.
24         BUG: se cae cuando el historial esta vacio."""
25         nuevo = Nodo(turno, modulo)
26         if self.cabeza is None: #el if revisa si la cabeza es None, si lo es, significa que no hay elementos en la lista y se agrega el nuevo
27             self.cabeza = nuevo # Si no es None, se recorre la lista hasta encontrar el último nodo y se agrega el nuevo nodo al final
28         else:
29             actual = self.cabeza
30             while actual.siguiente is not None:
31                 actual = actual.siguiente
32             actual.siguiente = nuevo
33
34     #corregir la funcion deshacer_ultima
35     def deshacer_ultima(self):
36         """Elimina la ULTIMA atencion registrada.
37         Devuelve True si elimino algo, False si el historial estaba vacio."""
38         if self.cabeza is None: # Si el historial esta vacio, devuelve False.
39             return False
40         if self.cabeza.siguiente is None: # Si solo hay un elemento en la lista, se elimina la cabeza y se devuelve True.
41             self.cabeza = None
42             return True
43         actual = self.cabeza
44         while actual.siguiente.siguiente is not None: #si hay mas de un elemento en la lista, se recorre hasta el penultimo
45             actual = actual.siguiente
46         actual.siguiente = None # Se elimina el último nodo
47         return True
48
49     def buscar(self, turno):
50         """Devuelve el modulo que atendio ese turno, o None si no existe.
51         PENDIENTE: implementar."""
52         actual = self.cabeza
53         while actual is not None:
54             if actual.turno == turno:
55                 return actual.modulo
56             actual = actual.siguiente
57         return None
58
59     def cuantas(self):
60         n = 0
61         actual = self.cabeza
62         while actual is not None:
63             n += 1
64             actual = actual.siguiente
65         return n
66
67     def listar(self):
68         r = []
69         actual = self.cabeza
70         while actual is not None:
71             r.append(actual.turno)
72             actual = actual.siguiente
73         return r
74

```

TCK-5512 · verificacion

```

[OK ] registrar en historial VACIO: obtuvo [101]
[OK ] registrar cuatro atenciones: obtuvo [101, 102, 103, 104]
[OK ] deshacer la ULTIMA: obtuvo [101, 102, 103]
[OK ] deshacer con UN solo elemento: obtuvo 0
[OK ] deshacer en historial VACIO: obtuvo False
[OK ] buscar turno existente: obtuvo A
[OK ] buscar turno inexistente: obtuvo None
[OK ] deshacer dos veces seguidas: obtuvo [101, 102]

```

TICKET CERRADO – codigo de cierre: 5512-34C

PS C:\Users\eeceg\OneDrive\Documentos\GitHub\Edward-Castillo>

En la línea 26 a la 29 se agrega un if con su else

En la línea 39 se agregan los condicionales para lograr que la función funcione y esta comentada en el código

En la línea 55 se agregan los condicionales para lograr que la función funcione y esta comentada en el código

Hoja de ruta del turno

Anote aquí el código de cierre de cada ticket que logre cerrar. Este es el documento que se entrega en la mesa de despacho.

Ticket	Sev	Sistema	Código de cierre obtenido	Puntos	Visto bueno
TCK-5510	P3	RedAcopio		1.5	
TCK-5511	P2	PrestaLab		1.7	
TCK-5512	P0	TurnoJusto		1.8	

Hora de cierre de turno: _____ Total de puntos: _____

Firma de despacho: _____

Antes de entregar, verifique

- Escribió su nombre en la portada.
- Copió cada código de cierre completo, tal como aparece en pantalla.
- Dejó los archivos guardados en su equipo, sin borrar nada.
- Tiene las pruebas listas para ejecutarlas delante de la profesora.